

Beta regression simulations

The Beta distribution

The Beta distribution is parameterized by α and β , called shape parameters. These are positive numbers and related to the central tendency μ and inverse dispersion (or precision¹), ϕ of the distribution:

$$\alpha = \mu \times \phi$$
$$\beta = (1 - \mu) \times \phi$$

μ is modelled on the logit scale where η is the linear predictor. To get μ from η we apply the inverse logit transformation:

$$\mu = \frac{\exp(\eta)}{1 + \exp(\eta)}$$

The inverse dispersion parameter ϕ is modeled on the log scale:

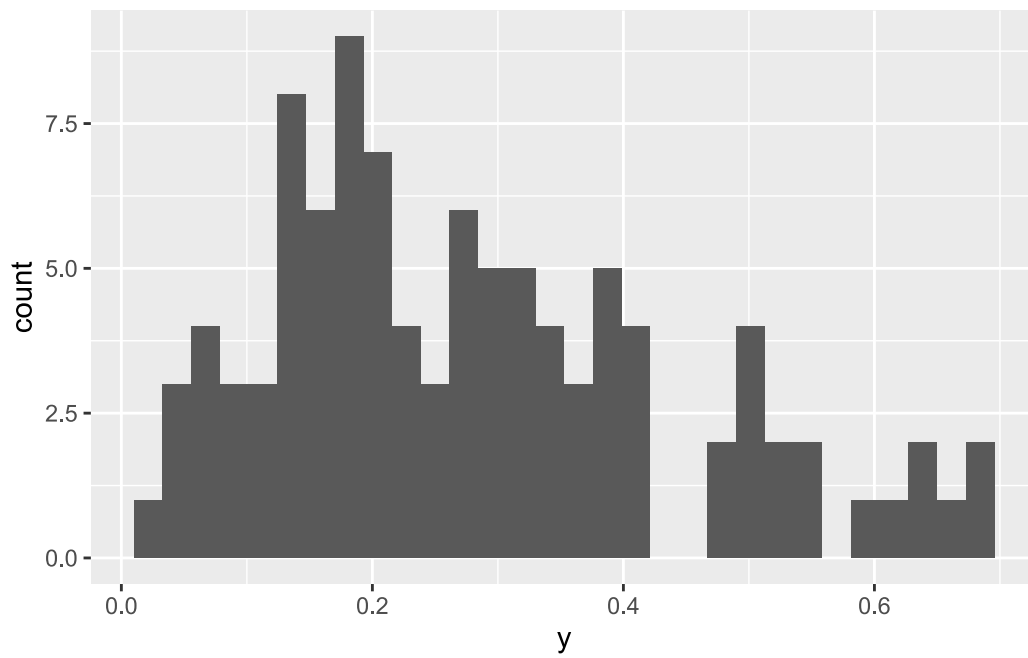
$$\phi = \exp(1/\sigma)$$

To simulate values from the beta distribution we can use `rbeta()`, e.g.:

```
library(tidyverse)
library(gt)

y <- rbeta(100, 2, 5)

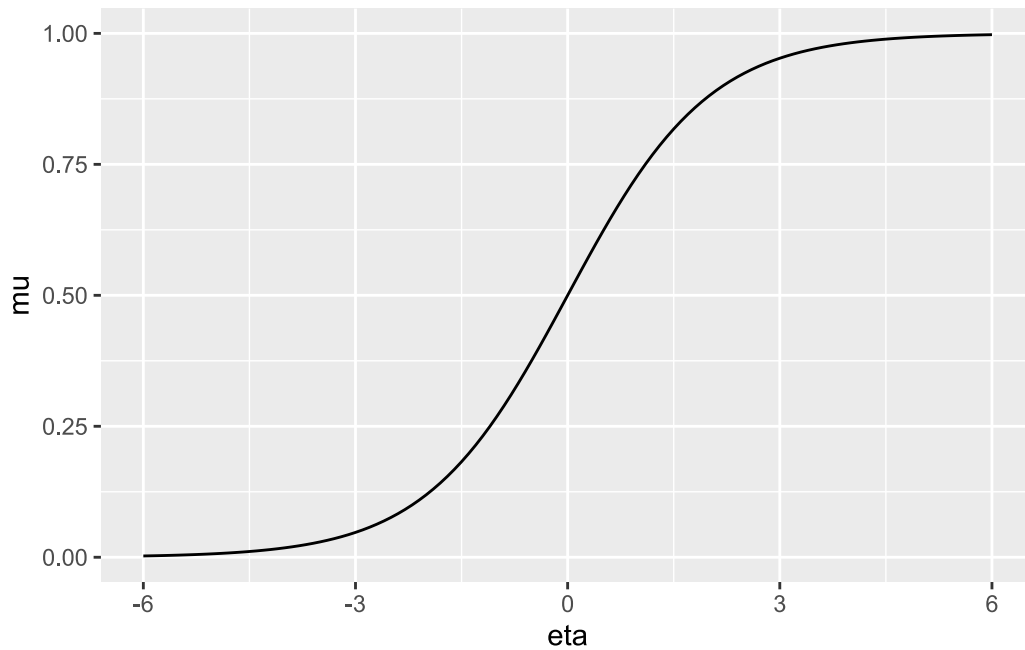
data.frame(y) |>
  ggplot(aes(y)) + geom_histogram()
```



The mean of the Beta distribution (μ) is related to the linear predictor, η , as shown below.

```
eta <- seq(from = -6, to = 6, by = 0.1)

data.frame(eta, mu = plogis(eta)) |>
  ggplot(aes(eta, mu)) + geom_line()
```



Simulating beta-distributed data

In our study we want to explore the effect of age (a continuous variable) and training status (a factor variable) on the proportion of retained introns (IR; $IR \in [0, 1]$). We are doing this in a repeated design, meaning that the same participants are evaluated before and after training (time). We add a varying intercept ($b_0 \sim \text{Normal}(0, \sigma_b)$) to the linear predictor to induce correlations in the data set.

$$IR \sim \text{Beta}(\alpha, \beta)$$

$$\alpha = \mu \times \phi$$

$$\beta = (1 - \mu) \times \phi$$

$$\mu = \frac{\exp(\eta)}{1 + \exp(\eta)}$$

$$\phi = \exp(1/\sigma)$$

$$\eta = \beta_0 + \beta_1 \times \text{age} + \beta_2 \times \text{training} + \beta_3 \times (\text{age} \times \text{time}) + b_0$$

$$\sigma = 2$$

$$b_0 \sim \text{Normal}(0, \sigma_b)$$

We can convert this model to a simulation function that will give us a data set of size N, balanced between groups.

```
ir_sim1 <- function(n = 10,
                    beta0 = 0,
```

```

        beta1 = -2,
        beta2 = 2,
        beta3 = -2,
        sigma = 2,
        sigma_b0 = 1) {

  if(!n %% 2 == 0) stop("n must be an even number")

  age <- c(rep(0, n/2), rep(1, n/2))
  age <- rep(age, 2)
  training <- c(rep(0, n), rep(1, n))
  b0 <- rnorm(n, 0, sigma_b0)
  b0 <- rep(b0, 2)

  eta <- beta0 + beta1 * age + beta2 * training + beta3 * age * training + b0

  phi <- exp(sigma)
  mu <- exp(eta) / (1 + exp(eta))

  alpha <- mu * phi
  beta <- (1 - mu) * phi

  y <- rbeta(n*2, alpha, beta)

  df <- data.frame(age, training, id = rep(1:n, 2), y = y)
  return(df)
}

```

The raw data from the simulation function produces outcome on the scale $y \in [0, 1]$ based on the parameters for the linear predictor of η .

```

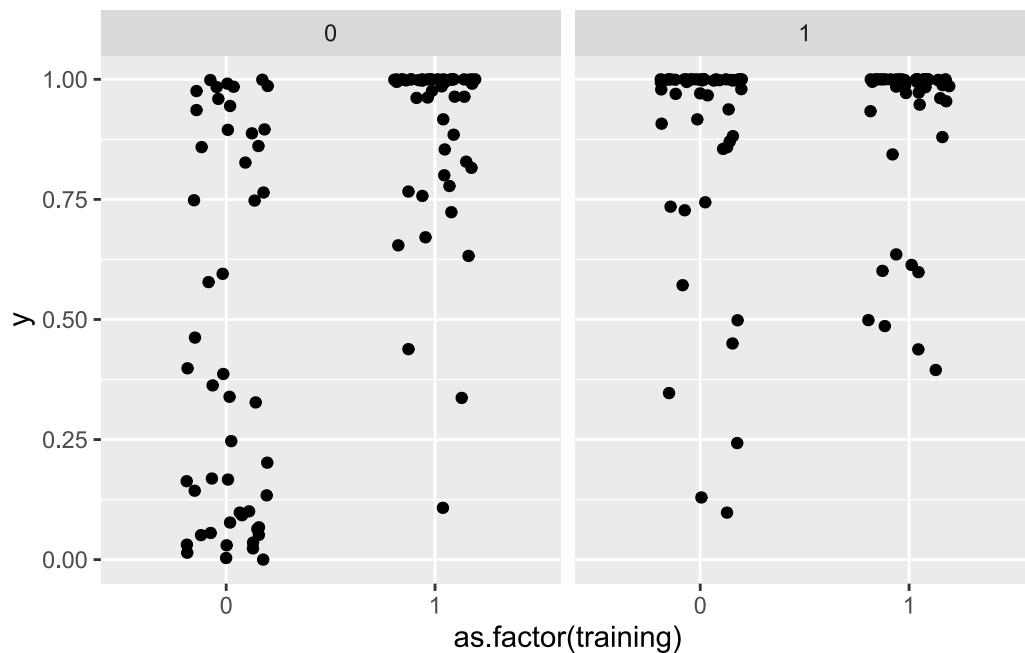
## Test the function

dat <- ir_sim1(n = 100, beta0 = 0, beta1 = 2, beta2 = 2, beta3 = -2, sigma =
0.1, sigma_b0 = 0.05)

dat |>
  ggplot(aes(as.factor(training), y)) +

```

```
geom_point(position = position_jitter(width = 0.2)) +
facet_wrap(~ age)
```



The `rbeta` function returns values in the interval $[0, 1]$ including boundaries (0, and 1). However, modelling with the `beta_family` distribution does not allow for boundary values (0,1). We have to compress the data prior to fitting.

```
compress <- function(x, e = 10^-4, additive = TRUE) {
  n <- length(x)

  if(additive) {
    xout <- x * (1 - 2*e) + e
  } else {
    xout <- (x * (n - 1) + 0.5) / n
  }

  xout
}

dat$y2 <- compress(dat$y)
```

```
library(glmTMB)

m <- glmTMB(y2 ~ age + training + age:training + (1|id), data = dat,
            family = "beta_family")

summary(m)
```

```
Family: beta ( logit )
Formula:          y2 ~ age + training + age:training + (1 | id)
Data: dat

      AIC      BIC    logLik -2*log(L)  df.resid
    -834.4    -814.6     423.2    -846.4      194

Random effects:

Conditional model:
Groups Name      Variance Std.Dev.
id (Intercept) 1.708e-09 4.133e-05
Number of obs: 200, groups: id, 100

Dispersion parameter for beta family (): 1.29

Conditional model:
              Estimate Std. Error z value Pr(>|z|)
(Intercept)  -0.1048    0.1726  -0.607    0.544
age           1.8356    0.2506   7.325 2.39e-13 ***
training      1.8445    0.2506   7.360 1.84e-13 ***
age:training  -1.7202    0.3349  -5.136 2.81e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The simulation function together with compression seems to return a data sets that we are able to model. In the construction of the parameters of the Beta distribution (α and β), the dispersion parameter ϕ is of importance. We will likely not get the coefficients from η back from the model due to ϕ . To explore the effect of ϕ :

```

sigmas <- seq(from = 0.1, to = 10, by = 0.2)

results <- list()

for(i in seq_along(sigmas)) {

  # Simulate data
  dat <- ir_sim1(n = 100, beta0 = 3, beta1 = -3, beta2 = -3, beta3 = 3, sigma =
sigmas[i], sigma_b0 = 0.25)

  dat$y2 <- compress(dat$y)

  # Fit model
  m <- glmmTMB(y2 ~ age + training + age : training, data = dat,
              family = "beta_family")

  results[[i]] <- data.frame(param = c("b0", "b1", "b2", "b3"),
                            coef = fixef(m)$cond,
                            sigma = sigmas[i],
                            row.names = NULL)

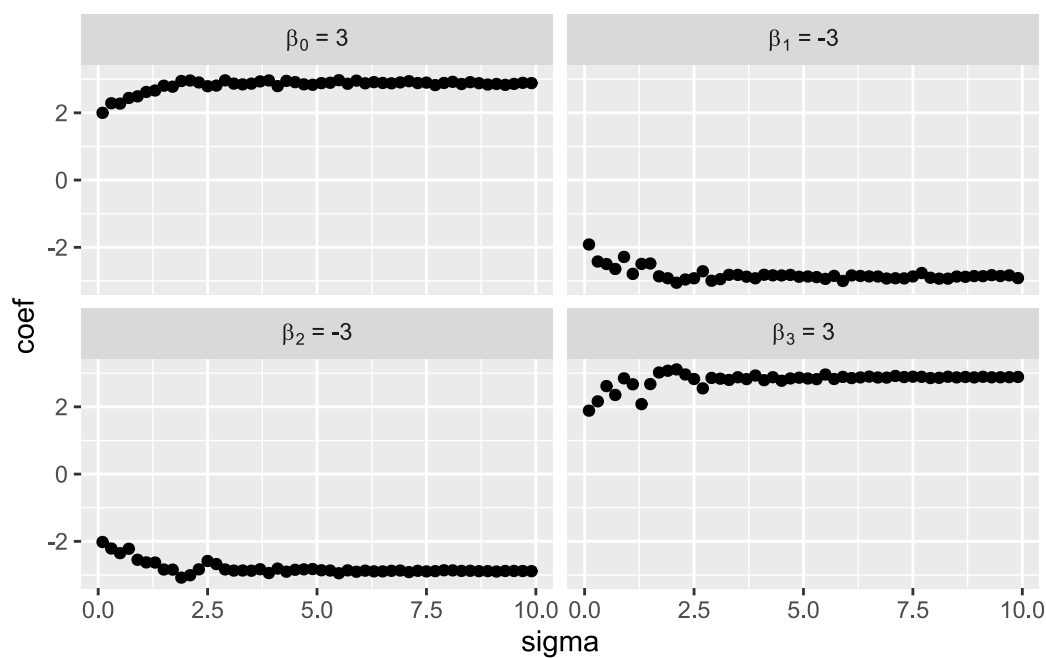
}

results <- bind_rows(results)

results |>
  mutate(eta_beta = rep(c('beta[0]*" = 3"',
                          'beta[1]*" = -3"',
                          'beta[2]*" = -3"',
                          'beta[3]*" = 3"'), length.out = length(sigmas) * 4))
|>

ggplot(aes(sigma, coef)) + geom_point() +
facet_wrap(~ eta_beta, labeller = label_parsed)

```



As ϕ increases, the precision increases. This means that we should think about σ in the simulation as being specified as $1/\sigma$. A large σ will result in small precision (and larger variation in the coefficients).

Specifying predictions and contrasts

Using the `marginalEffects` package we can do predictions and compute contrasts. This can be useful for visualizations and inference. Below we first specify a contrast for the design we've simulated above and then apply it as a function in a `seqwrap` call.

In `marginalEffects::predictions` we can specify a data grid from where we want predictions. Using the `datagrid` function we can specify the available variables in the data set, age and training, using the `unique` function. The table shows the output, we have specified "response" as the scale on which we want the predictions.

```
# Load marginalEffects
library(marginalEffects)
```

```
Attaching package: 'marginalEffects'
```

```
The following object is masked from 'package:glmmTMB':
```

```
refit
```

```

# Simulate data and compress the observations
dat <- ir_sim1(n = 100, beta0 = 3, beta1 = -3, beta2 = -3, beta3 = 3, sigma =
3, sigma_b0 = 0.5)

dat$y2 <- compress(dat$y)

# Fit model
m <- glmmTMB(y2 ~ age + training + age : training, data = dat,
             family = "beta_family")

# Do predictions
preds <- marginalesffects::predictions(
  m, # The model object
  re.form = NA, # The form of the random effects - This gives us the average
per group
  type = "response", # Predict on the response scale (observed scale)
  # Use the data from the model data grid
  newdata = marginalesffects::datagrid(
    training = unique,
    age = unique))

preds |>
  gt() |>
  fmt_auto() |>
  opt_table_font(size = 10)

```

Table 1: Output from a marginalesffects prediction.

rowid	estimate	std.error	statistic	p.value	s.value	conf.low	conf.high	y2	training	age	df
1	0.922	0.01	95.155	0	Inf	0.903	0.941	0.578	0	0	Inf
2	0.471	0.022	21.285	1.550×10^{-100}	331.56	0.428	0.515	0.578	0	1	Inf
3	0.459	0.022	20.783	6.122×10^{-96}	316.291	0.416	0.503	0.578	1	0	Inf
4	0.434	0.022	19.753	7.588×10^{-87}	286.084	0.391	0.477	0.578	1	1	Inf

We can use the prediction data table to specify contrasts as hypotheses. In the `hypothesis` slot we list a key (e.g., `"train_old"`) and a contrast as differences between rows in the prediction table (e.g., the first row corresponding to `training == 0` and `age == 0` is `"b1"`). The hypothesis is `= 0` meaning that the null-hypothesis is that there are no differences. The `predictions` function will handle the SE calculation and create corresponding confidence intervals.

```

# Disable a warning message regarding specifying contrasts
options(marginaleffects_safe = FALSE)

contr <- marginaleffects::predictions(
  m, # The model object
  re.form = NA, # The form of the random effects - This gives us the average
per group
  type = "response", # Predict on the response scale (observed scale)
  # Use the data from the model data grid
  newdata = marginaleffects::datagrid(
    training = unique,
    age = unique),
  hypothesis = c(
    # Training effect within old
    "train_old" = "b3 - b1 = 0",
    # Training effect within young
    "train_young" = "b4 - b2 = 0",
    # Training effect young vs old
    "train_old_young" = "(b3 - b1) - (b4 - b2) = 0"
  ))

```

The plot below shows the simulated data, predictions and contrasts.

```

# Plot 1: Predictions
p1 <- dat |>
  ggplot(aes(as.factor(training), y)) +
  geom_point(position = position_jitter(width = 0.1)) +

  geom_errorbar(data = data.frame(preds),
    aes(as.factor(training), estimate,
      ymin = conf.low, ymax = conf.high),
    width = 0.2) +

  geom_point(data = data.frame(preds),
    aes(as.factor(training), estimate),
    shape = 21, size = 4, fill = "orange") +

  facet_wrap(~ age)

# Plot 2: Contrasts
p2 <- contr |>
  data.frame() |>

```

```

ggplot(aes(hypothesis, estimate)) +
  geom_errorbar(aes(ymin = conf.low, ymax = conf.high),
               width = 0.1) +
  geom_point(shape = 21, fill = "steelblue",
            size = 4)

cowplot::plot_grid(p1, p2, ncol = 1)

```

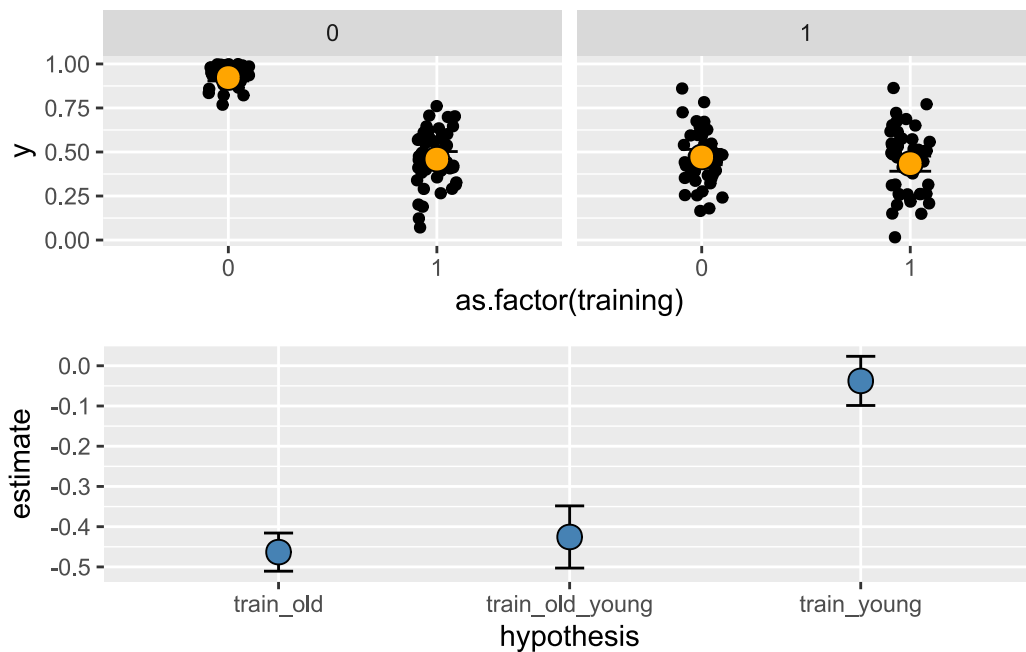


Figure 1: Simulated data points (black), predictions (orange) and contrasts (blue) with 95% confidence intervals.

Next we can define a function to get the marginal predictions out from a seqwrap model object. To showcase this, first we need a simulated data set and metadata.

```

# A single data set with random combinations of parameter values can be simulated
as
dat <- ir_sim1(n = 80, beta0 = rnorm(1,0,2),
              beta1 = rnorm(1,0,3), beta2 = rnorm(1,0,3),
              beta3 = rnorm(1,0,1), sigma = rnorm(1,0,3),
              sigma_b0 = abs(rnorm(1,0.4, 0.2)))

# We can wrap it in a function ...
sim_wrap <- function() {
  dat <- ir_sim1(n = 80, beta0 = rnorm(1,0,2), beta1 = rnorm(1,0,3),

```

```

beta2 = rnorm(1,0,3), beta3 = rnorm(1,0,1), sigma = rnorm(1,0,3), sigma_b0 =
abs(rnorm(1,0.4, 0.2)))

dat$y2 <- compress(dat$y)

return(dat)
}

# And simulate multiple data sets
# Return N simulations and collect them in a data frame
sims <- replicate(100, sim_wrap(), simplify = FALSE)
sims_df <- do.call(rbind, lapply(sims, function(df) t(df[, 5, drop = FALSE])))
|>
  data.frame(row.names = NULL)

# Set seq_sample_id (reuse dat from above)
colnames(sims_df) <- paste0("s", dat$id, "_", dat$training)

sims_df$target <- paste0("target", 1:100)

sims_df <- sims_df |>
  dplyr::select(target, starts_with("s"))

metadata <- dat |>
  mutate(seq_sample_id = paste0("s", id, "_", training) ) |>
  dplyr::select(id, age, training, seq_sample_id)

```

We now specify the `marginaleffects` prediction-contrast-function as a summary function. Notice that I'm adding predictions to the hypothesis slot. **Importantly** when using the `marginaleffects` package in `seqwrp` we need to specify in the function that warnings should be disabled.

```

sum_fun <- function(m) {

  # Disable a warning message regarding specifying contrasts
  options(marginaleffects_safe = FALSE)

  contr <- marginaleffects::predictions(
    m, # The model object
    re.form = NA, # The form of the random effects - This gives us the average
    # per group
    type = "response", # Predict on the response scale (observed scale)
    # Use the data from the model data grid

```

```

newdata = marginalesffects::datagrid(
  training = unique,
  age = unique),
hypothesis = c(
  "old_baseline" = "b1 = 0",
  "young_baseline" = "b2 = 0",
  "old_post" = "b3 = 0",
  "young_post" = "b4 = 0",

  # Training effect within old
  "train_old" = "b3 - b1 = 0",
  # Training effect within young
  "train_young" = "b4 - b2 = 0",
  # Training effect young vs old
  "train_old_young" = "(b3 - b1) - (b4 - b2) = 0"
)) |>
  data.frame()

return(contr)
}

```

The seqwrap setup is constructed as in the vignette

```

library(seqwrap)

m1 <- seqwrap_compose(
  data = sims_df,
  metadata = metadata,
  modelfun = glmmTMB::glmmTMB,
  arguments = alist(
    formula = y ~ age * training + (1|id),
    family = glmmTMB::beta_family
  ),
  summary_fun = sum_fun)

mlr <- seqwrap(m1,
  cores = 10,
  return_models = FALSE)

```

```
—  
seqwrap
```

```
Initiating clusters for parallel processing with 10 cores  
Merging and modelling data  
jun 05 2026 15:20:10: Completed model fitting and evaluation. Elapsed time was  
0.36 minutes
```

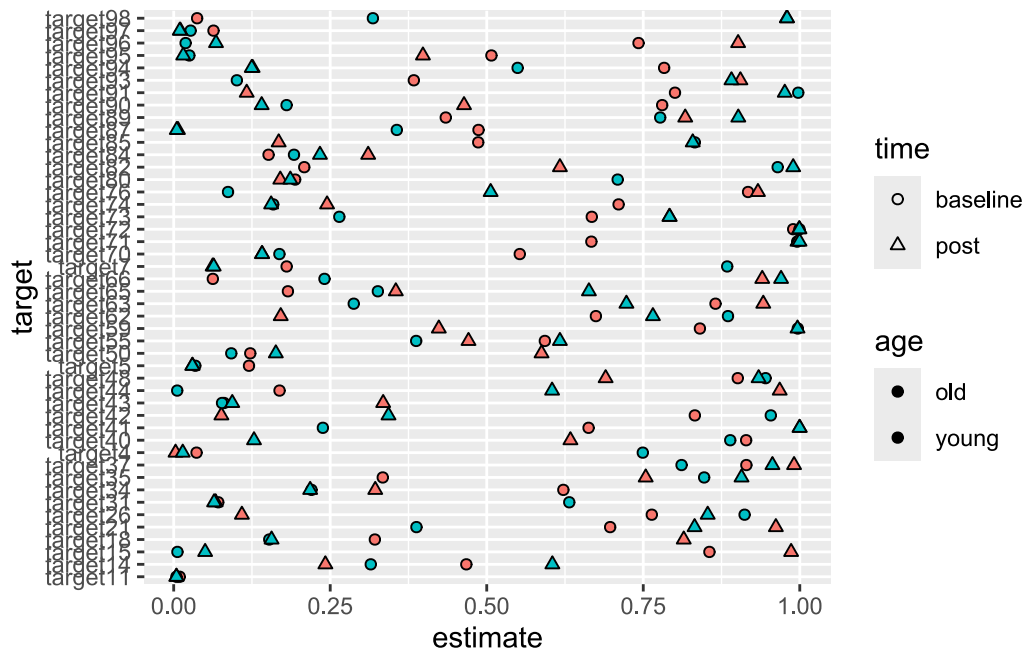
```
m1s <- seqwrap_summarise(m1r)
```

```
—  
summarise — seqwrap  
• A total of 160 samples and 100 targets where used in `glmmTMB::glmmTMB` with  
arguments `alist(formula = y ~ age * training + (1 | id), family =  
glmmTMB::beta_family)`  
• Attempting to combine results from the provided summary function.  
• Attempting to combine results from the provided evaluations function.  
• Attempting to summarise errors and warnings from the fitting process.  
  
—  
summaries — Model  
• 100 targets have associated summaries  
  
—  
evaluations — Model  
• 100 targets have associated evaluation results  
  
—  
results — Combined  
i Combined results have been generated and were  
silently returned.
```

Now we have a seqwrap summary with contrasts and predictions specified using `marginalEffects`. These can of course be plotted with the outcome on the response scale.

```
# Plotting targets with significant interaction effect  
targets <- m1s$summaries |>  
  filter(hypothesis == "train_old_young" & p.value < 0.01) |>  
  pull(target)
```

```
mls$summaries |>
  filter(target %in% targets) |>
  filter(grepl("baseline|post", hypothesis)) |>
  separate(hypothesis, into = c("age", "time")) |>
  ggplot(aes(estimate, target, fill = age, shape = time)) + geom_point() +
  scale_shape_manual(values = c(21, 24))
```



Multilevel Beta in a meta-analysis context

How can we interpret coefficients in the multilevel version of the Beta model?

Interpretation of our data will benefit from interpretation using a multilevel model. Data comes from a number of different studies, with multiple samples from participants in each study.

```
# beta0, global intercept
# beta1, global slope for age
# beta2, global training effect
# sigma0/sigma1/sigma2, SD of global effects between studies
# sigma_participant, SD of between participant intercepts
# sigma, "residual" variation should be entered as precision 1/sigma
# age/training/study/participants, design variables
#
#
# returns a vector of simulated IR values based on the design variables.
ir_sim2 <- function(beta0 = 3,
```

```

    beta1 = -3,
    beta2 = 3,
    sigma0 = 1,
    sigma1 = 1,
    sigma2 = 1,
    sigma = 1/0.5,
    sigma_participant = 1,
    age = c(1, 2, 1, 2),
    training = c(0,0,0,0),
    study = c(1, 2, 3, 4),
    participant = c(1, 2, 4, 5)) {

# Save n observations
n <- length(participant)
# Check that all design variables are of equal length
  if(length(unique(lengths(list(participant, study, training, age)))) != 1)
stop("All design variables (age, training, study, participant) must be of the
same length.")

# List all studies
studies <- unique(study)

## Draw intercepts per study
beta0s <- rnorm(length(studies), beta0, sigma0)

## Draw age-slopes per study
betas <- rnorm(length(studies), beta1, sigma1)

## Draw training slopes per study
beta2s <- rnorm(length(studies), beta2, sigma2)

## List all participants
participants <- unique(paste0(participant, "_", study))

## Draw intercept per participant
b0s <- rnorm(length(participants), 0, sigma_participant)

# Combine all drawn values

partdat <- data.frame(participant_study = participants,
                      b0 = b0s) |>
  tidyr::separate(participant_study,

```

```

      into = c("participant", "study"),
      sep = "_")

studydat <- data.frame(study = as.character(studies),
                      beta0 = beta0s,
                      beta1 = beta1s,
                      beta2 = beta2s)

eta <- partdat |>
  dplyr::inner_join(studydat, by = "study") |>
  dplyr::inner_join(data.frame(age,
                              training,
                              study = as.character(study),
                              participant = as.character(participant))) |>
  dplyr::mutate(eta = beta0 + beta1 * age + beta2 * training + b0) |>
  dplyr::pull(eta)

phi <- exp(sigma)
mu <- exp(eta) / (1 + exp(eta))

alpha <- mu * phi
beta <- (1 - mu) * phi

y <- rbeta(n, alpha, beta)

return(y)
}

```

Multilevel Beta regression with simulated data

```

design <- rbind(
  # Study 1
  data.frame(study = 1,
             participant = rep(1:20, each = 2),
             age = rep(runif(20, 0.5, 1), each = 2),
             training = c(0,1)),
  # Study 2
  data.frame(study = 2,
             participant = rep(1:20, each = 2),
             age = rep(runif(20, 0.1, 0.6), each = 2),
             training = c(0,1)),
  # Study 3

```

```

data.frame(study = 3,
            participant = rep(1:20, each = 2),
            age = rep(runif(20, 0.1, 0.9), each = 2),
            training = c(0,1)),
# Study 4
data.frame(study = 4,
            participant = rep(1:20, each = 2),
            age = rep(runif(20, 0.01, 0.3), each = 2),
            training = c(0,1)),
# Study 5
data.frame(study = 5,
            participant = rep(1:20, each = 2),
            age = rep(runif(20, 0.01, 0.99), each = 2),
            training = c(0,1))
)

```

Next we simulate (and compress) the data.

```

y <- ir_sim2(beta0 = 3, beta1 = -1.5, beta2 = 2,
             sigma0 = 2, sigma1 = 1, sigma2 = 2,
             sigma = 1/0.5,
             sigma_participant = 0.66,
             age = design$age,
             training = design$training,
             participant = design$participant,
             study = design$study)

```

Joining with `by = join\_by(participant, study)`

```

dat1 <- design |>
  mutate(participant = paste0(study, "_", participant))

dat1$y <- compress(y)

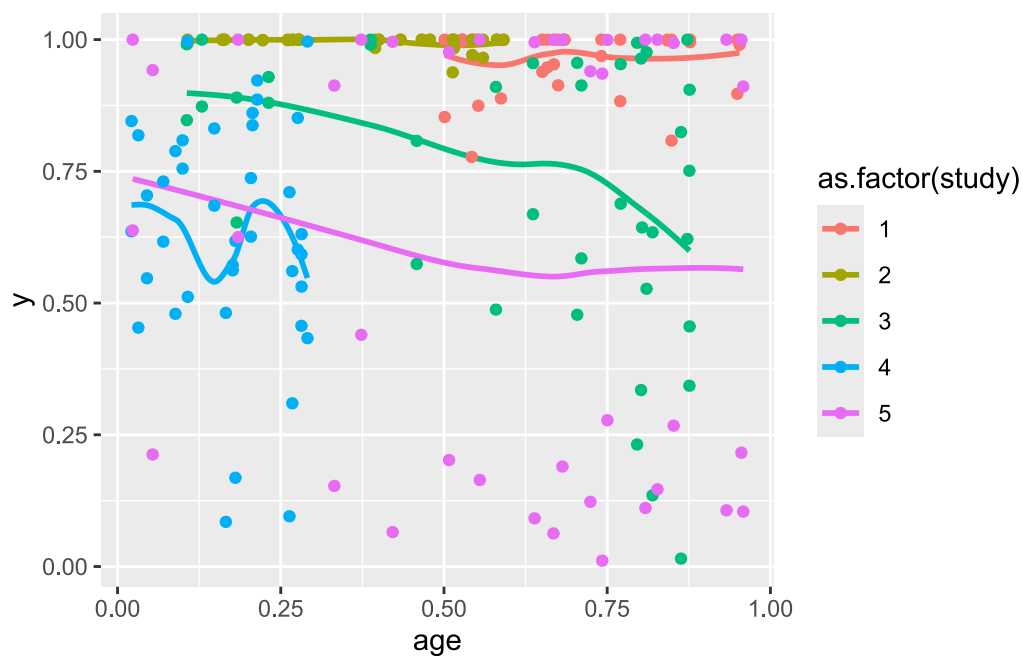
```

```

dat1 |>
  ggplot(aes(age, y,
             color = as.factor(study))) +
  geom_smooth(se = FALSE) +
  geom_point()

```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



```
m <- glmmTMB(y ~ age * training +
              (1|study) + (1|participant),
              data = dat1,
              family = beta_family() )
```

```
summary(m)
```

```
Family: beta ( logit )
Formula:      y ~ age * training + (1 | study) + (1 | participant)
Data: dat1
```

AIC	BIC	logLik	-2*log(L)	df.resid
-898.3	-875.2	456.1	-912.3	193

Random effects:

Conditional model:

Groups	Name	Variance	Std.Dev.
study	(Intercept)	1.246e+00	1.1161380
participant	(Intercept)	2.430e-09	0.0000493

Number of obs: 200, groups: study, 5; participant, 100

Dispersion parameter for beta family (): 3.28

Conditional model:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	1.8478	0.5536	3.337	0.000845	***
age	-2.0687	0.4244	-4.874	1.09e-06	***
training	0.6517	0.2893	2.253	0.024281	*
age:training	2.2731	0.5032	4.517	6.27e-06	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

VarCorr(m)

Conditional model:

Groups	Name	Std.Dev.
study	(Intercept)	1.1161e+00
participant	(Intercept)	4.9296e-05

Bibliography

- 1 Douma JC, Weedon JT. Analysing Continuous Proportions in Ecology and Evolution: A Practical Introduction to Beta and Dirichlet Regression. *Methods in Ecology and Evolution* 2019; **10**: 1412–30.